

ASSIGNMENT-3.4

Name: Peesari Naveen
HT.No: 2303A51339
Batch: 20

Task 1: Zero-shot Prompt – Fibonacci Series Generator

Scenario : In this task, a zero-shot prompting technique was used. A single comment prompt was written without providing any examples, instructing GitHub Copilot to generate a Python function that prints the first N Fibonacci numbers.

Prompt: # Write a Python function to print the first N Fibonacci numbers

Code:

Result:

Observation:

The zero-shot prompt was sufficient for Copilot to correctly infer the Fibonacci logic, even without any examples or additional context. However, the function behavior depended heavily on Copilot's prior training, and the output format was assumed rather than explicitly defined. This shows that zero-shot prompting works well for well-known problems but may lack consistency for ambiguous or complex tasks.

Task 2: One-shot Prompt – List Reversal Function

Scenario: This task used a one-shot prompting approach by providing a single example along with the instruction to help Copilot generate a correct list reversal function.

Prompt:

Write a Python function to reverse a list
Example: input [1, 2, 3] -> output [3, 2, 1]

Result:

Observation:

Providing one example significantly improved Copilot's accuracy and confidence in choosing an optimal approach. The generated solution was concise and efficient, using Python slicing. Compared to zero-shot prompting, one-shot prompting reduced ambiguity and guided Copilot toward the expected output format and logic.

Task 3: Few-shot Prompt – String Pattern Matching

Scenario: This task used a few-shot prompting technique by providing multiple examples to help Copilot understand a specific string validation pattern.

Prompt:

Write a function to check if a string starts with a capital letter and ends with a period
Example: "Hello." -> True
Example: "hello." -> False
Example: "Hello" -> False

Result:

Observation:

The few-shot prompt enabled Copilot to accurately identify the string pattern requirements and generate a precise validation function. The multiple examples clarified edge cases and reduced misinterpretation. This demonstrates that few-shot prompting is highly effective when pattern recognition or conditional logic is involved.

Task 4: Zero-shot vs Few-shot – Email Validator

Prompt: Zero-Shot Prompt:

Write a Python function to validate an email address

Prompt: Few-Shot Prompt:

Write a Python function to validate an email address

Example: "test@gmail.com" -> True

Example: "testgmail.com" -> False

Example: "test@com" -> False

Observation:

The zero-shot version produced a very basic and unreliable validation logic, while the few-shot prompt resulted in a more structured and realistic solution. The examples guided Copilot to include domain checks and input validation, significantly improving reliability.

Task 5: Prompt Tuning – Summing Digits of a Number

Prompt:

Style-1: Generic Prompt

Write a function to return the sum of digits of a number

Prompt with I/O Example:

Write a function to return the sum of digits of a number

Example: sum_of_digits(123) -> 6

Observation:

The prompt that included an input-output example produced a cleaner and more optimized implementation. The example encouraged Copilot to generate concise and Pythonic code using built-in functions. This demonstrates that prompt tuning with examples not only improves correctness but also enhances code quality and efficiency.